

Structure Programme C

```
#include <stdio.h> // 1-Bibliothèque et Constantes
#define Cont value

int main()
{
    // 2-Variables et Constantes

    // 3-Initialisation des Variables

    // 4-Operations

    // 5-Resultats

    return 0;
}
```

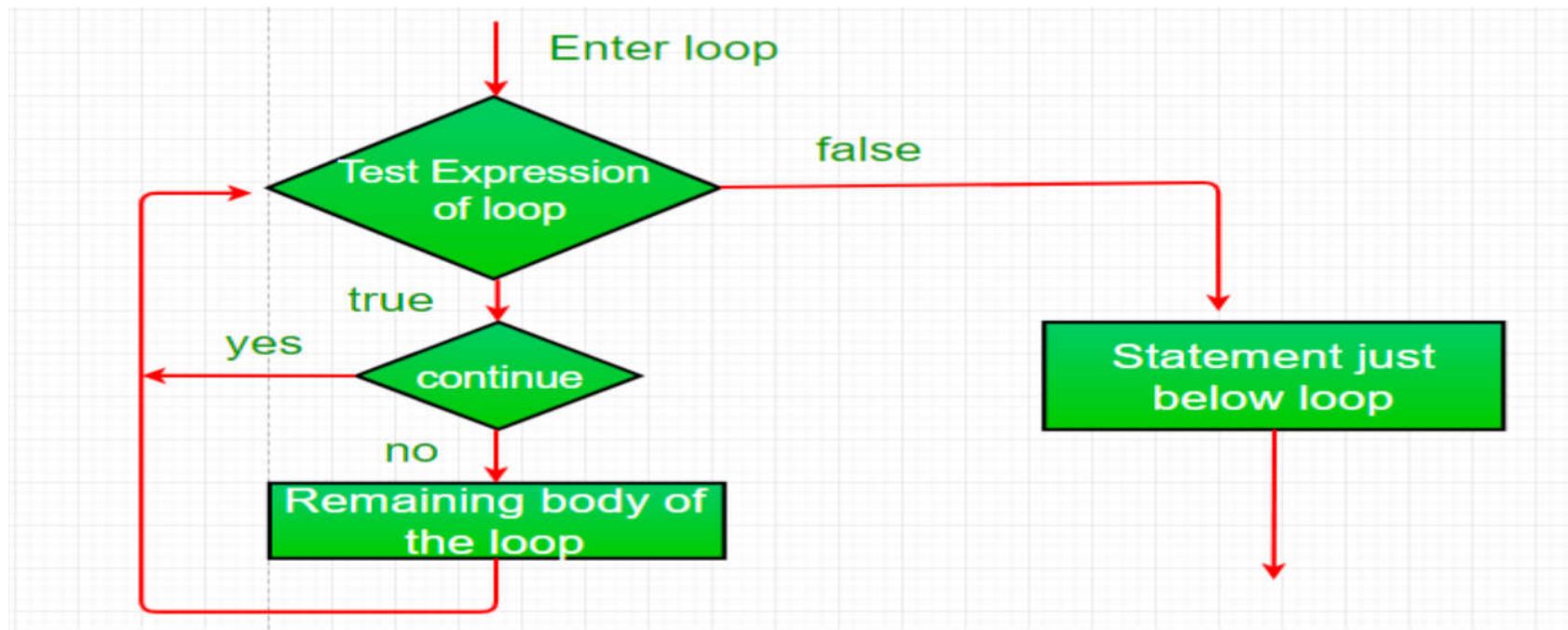
Ruptures de séquence

► Ruptures de séquence

- Dans le cas où une boucle commande l'exécution d'un bloc d'instructions, il peut être intéressant de vouloir sortir de cette boucle alors que la condition de passage est encore valide.
- Ce type d'opération est appelé une **rupture de séquence**.
- Les **ruptures de séquence** sont utilisées lorsque des conditions multiples peuvent conditionner l'exécution d'un ensemble d'instructions.
- Les ruptures de séquence sont réalisées par quatre instructions qui correspondent à leur niveau de travail :
 - Ruptures de **séquencement** de boucle : **continue** ;
 - Ruptures de **bloc**: **break**;
 - Ruptures **d'ordre de code** : **goto : Label** ;
 - Ruptures de **fonction**: **return value**;
 - Ruptures de **programme** : **exit(value)** / **<stdlib.h>**

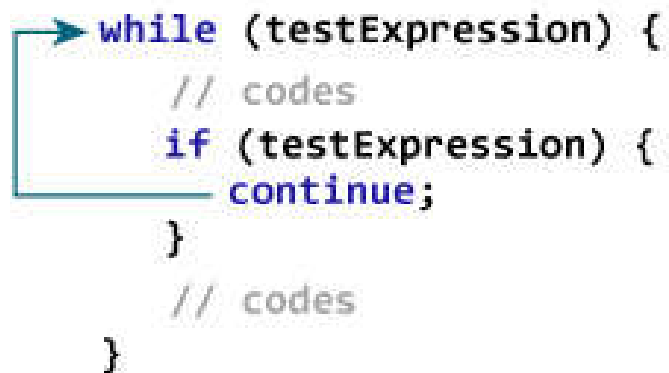
Ruptures de séquence : continue

- ▶ L'instruction **continue** est utilisée en relation avec les **boucles**.
- ▶ Elle provoque le passage à l'itération suivante de la boucle en **sautant à la fin du bloc**. Ce faisant, elle provoque la **non exécution des instructions** qui la suivent à l'intérieur du bloc.
- ▶ **Flowchart: continue**



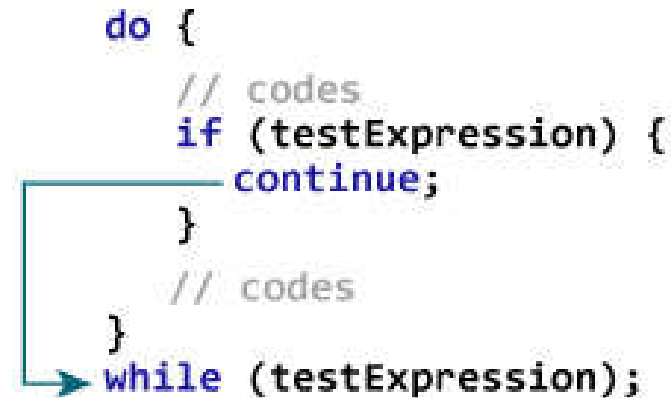
Ruptures de séquence : continue

► Flowchart: **continue**



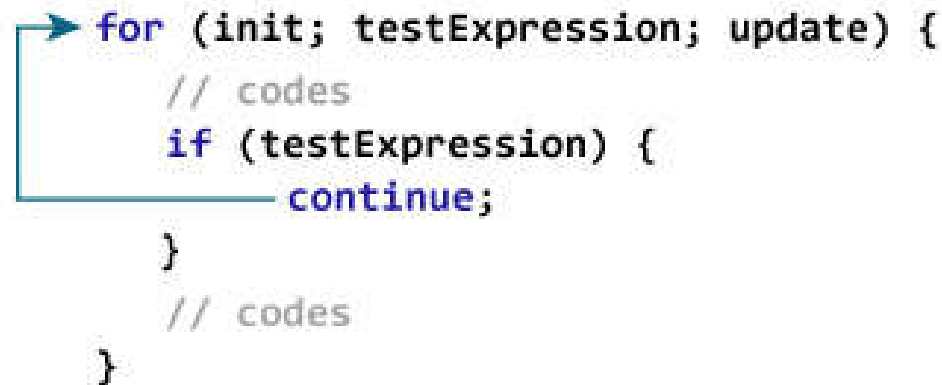
```
while (testExpression) {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
}
```

The flowchart shows a loop starting at the 'while' condition. If the condition is true, it enters the loop body. Inside the body, there is an 'if' statement. If the 'if' condition is true, the flow goes to the 'continue' statement, which loops back to the 'while' condition. If the 'if' condition is false, the flow goes to the next line of code in the loop body. After the loop body, the flow goes back to the 'while' condition.



```
do {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
} while (testExpression);
```

The flowchart shows a loop starting at the 'do' block. It enters the loop body. Inside the body, there is an 'if' statement. If the 'if' condition is true, the flow goes to the 'continue' statement, which loops back to the start of the 'do' block. If the 'if' condition is false, the flow goes to the next line of code in the loop body. After the loop body, the flow goes to the 'while' condition. If the condition is true, it loops back to the start of the 'do' block. If the condition is false, it exits the loop.



```
for (init; testExpression; update) {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
}
```

The flowchart shows a loop starting at the 'for' condition. If the condition is true, it enters the loop body. Inside the body, there is an 'if' statement. If the 'if' condition is true, the flow goes to the 'continue' statement, which loops back to the 'for' condition. If the 'if' condition is false, the flow goes to the next line of code in the loop body. After the loop body, the flow goes back to the 'for' condition.

Ruptures de séquence : continue

► continue

► Exemple :

```
1 // Program to calculate the sum of a maximum of 10 numbers
2 // Negative numbers are skipped from the calculation
3 #include <stdio.h>
4 int main()
5 {
6     int i;
7     double number, sum = 0.0;
8
9     for(i=1; i <=3; ++i)
10    {
11        printf("Enter a n%d: ",i);
12        scanf("%lf",&number);
13
14        if(number < 0.0)
15        {
16            printf("!!!!!!Nombre negatif!!!!!! \n");
17            continue; // = jump to next loop;
18        }
19
20        sum += number; // sum = sum + number;
21    }
22
23    printf("Sum = %.2lf",sum);
24
25    return 0;
26 }
```

► Exécution :

```
Enter a n1: 1
Enter a n2: -2
!!!!!!Nombre negatif!!!!!!
Enter a n3: -3
!!!!!!Nombre negatif!!!!!!
Sum = 1.00
```

Ruptures de séquence : continue

► Exercice :

```
int main()
{   int x=0;                                // initialisation
    while(x <10){                            // condition
        printf("%d\n",x);
        if(x==5) continue ;
        x++;                                // incrémentation
        printf("%d\n",x);
    }
    return 0;
}
```

► Exécution :

0 1 1 2 2 3 3 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5

Ruptures de séquence : continue

► Exercice :

```
#include <stdio.h>

int main()
{ int x =0;
  while(x<5)
  {
    printf("%d \t",x);
    x++;
    if(x==3) continue;
    printf("%d \n",x);
  }
  return 0;
}
```

► Exécution :

0	1	
1	2	
2	3	4
4	5	

Ruptures de séquence : continue

► Exercice :

```
#include <stdio.h>

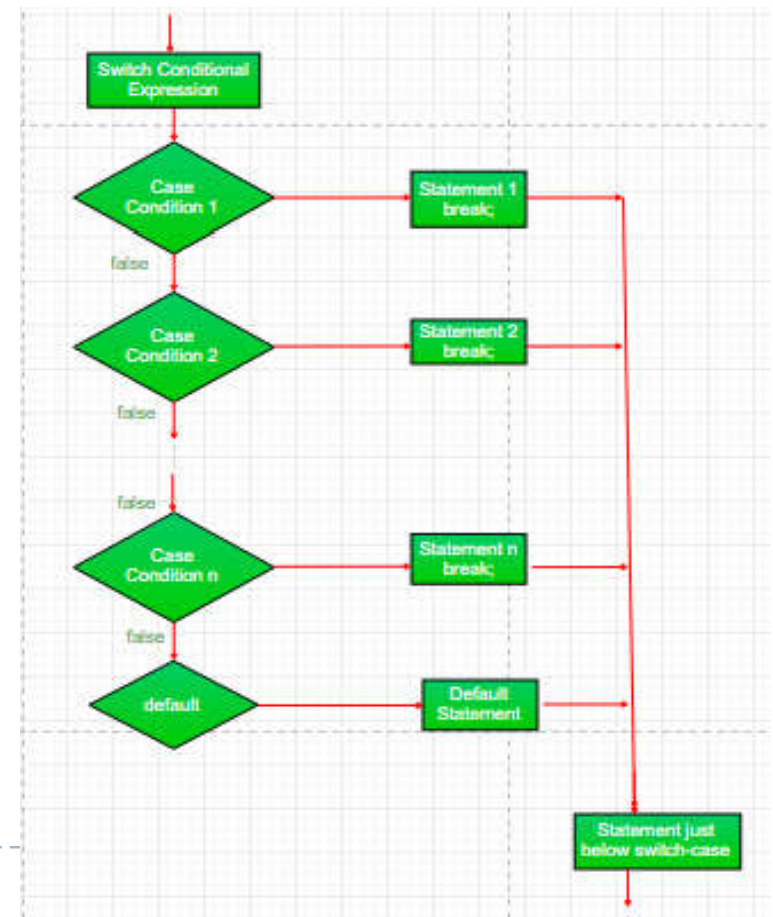
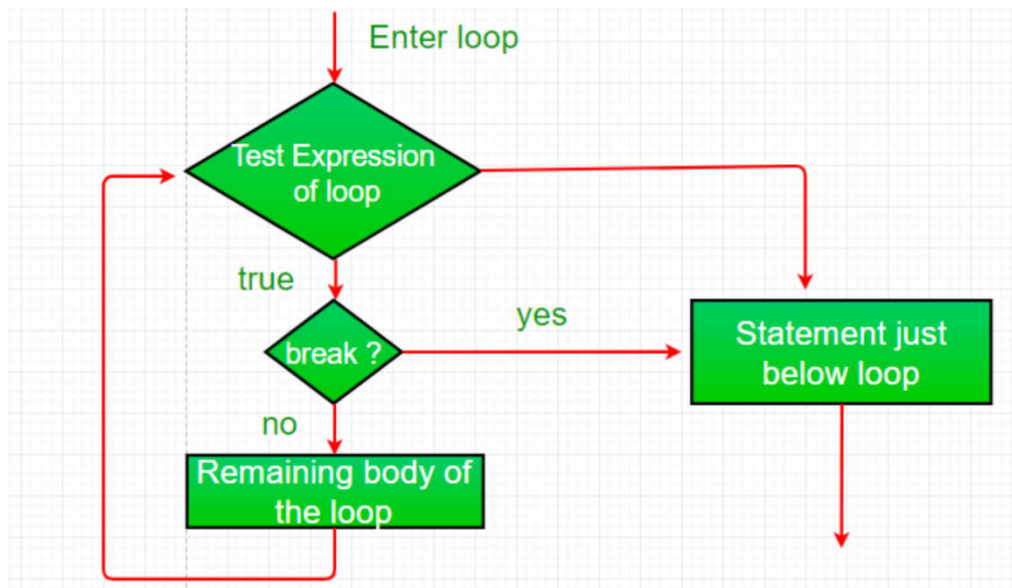
int main()
{ int x =0;
  while(x<5)
  {
    printf("%d \t",x);
    x++;
    if(x>3) continue;
    printf("%d \n",x);
  }
  return 0;
}
```

► Exécution :

0	1
1	2
2	3
3	4

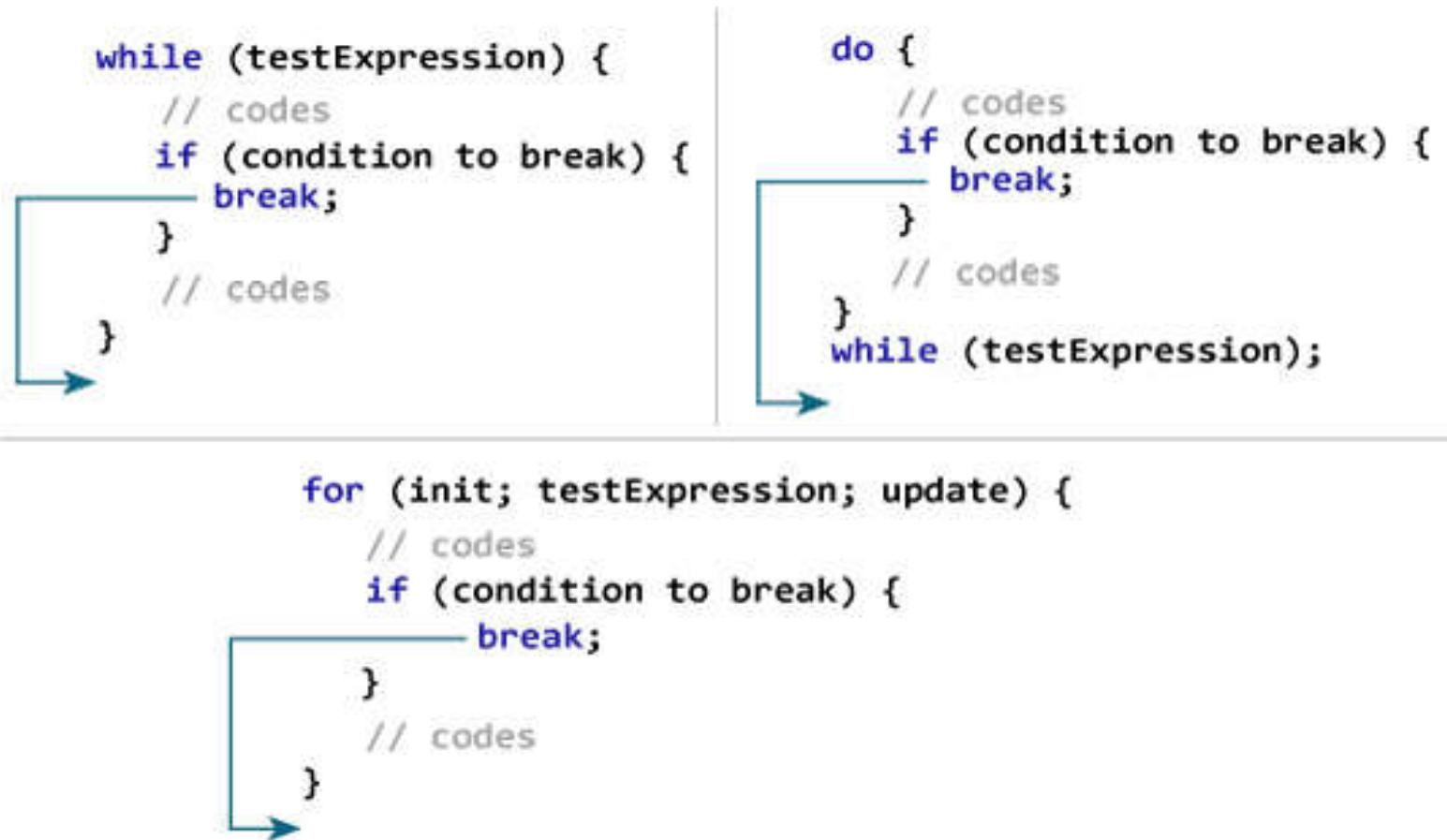
Ruptures de séquence : break

- ▶ L'instruction **break** est utilisé déjà dans le **switch**.
- ▶ L'instruction **break** permet de sortir d'un bloc d'instruction associé à une instruction **répétitive** ou **alternative** contrôlée par les instructions **switch**, **for**, un **while** ou un **do while**.
- ▶ **Flowchart: break continue**



Ruptures de séquence : continue

► Flowchart: **break**



Ruptures de séquence : break

► break

► Exemple : Boucle

```
1 // Program to calculate the sum of a maximum of 10 numbers
2 // If a negative number is entered, the loop terminates
3 #include <stdio.h>
4 int main()
5 {
6     int i;
7     double number, sum = 0.0;
8
9     for(i=1; i <=4; ++i)
10    {
11        printf("Enter a n%d: ",i);
12        scanf("%lf",&number);
13
14        // If the user enters a negative number, the loop ends
15        if(number < 0.0)
16        {
17            printf("!!!!Arret de Boucle!!!! \n");
18            break;
19        }
20
21        sum += number; // sum = sum + number;
22    }
23
24    printf("Sum = %.2lf",sum);
25
26    return 0;
27 }
```

► Exécution :

```
Enter a n1: 1
Enter a n2: 2
Enter a n3: -4
!!!!Arret de Boucle!!!!
Sum = 3.00
```

Ruptures de séquence : break

► break

► Exemple : Switch

```
1 #include <stdio.h>
2 int main()
3 {
4     int i;
5     printf("Enter a i: ");
6     scanf("%d",&i);
7     switch (i)
8     {
9         case 0:
10            printf("i is zero.");
11            break;
12        case 1:
13            printf("i is one.");
14            break;
15        case 2:
16            printf("i is two.");
17            break;
18        default:
19            printf("i is greater than 2.");
20    }
21    return 0;
22 }
```

► Exécution :

```
Enter a i: 0
i is zero.
```

```
Enter a i: 1
i is one.
```

```
Enter a i: 6
i is greater than 2.
```

Ruptures de séquence : break

► Exercice :

```
int main()
{
    int n;

    while (-1)
    {
        printf("Tapez un nombre Positif ! ");
        scanf("%d", &n);
        if(n>0) break;
    }

    printf(" Suite du Programme ");
    return 0;
}
```

► Exécution :

Ruptures de séquence : break

► Exercice :

```
int main()
{
    int n=0;

    while (n<5)
    {
        printf("nombre %d ",n);
        if(n==3) break;
    }

    printf(" Suite du Programme ");
    return 0;
}
```

► Exécution :

Ruptures de séquence : break

► Exercice :

```
int main()
{
    int n=0;

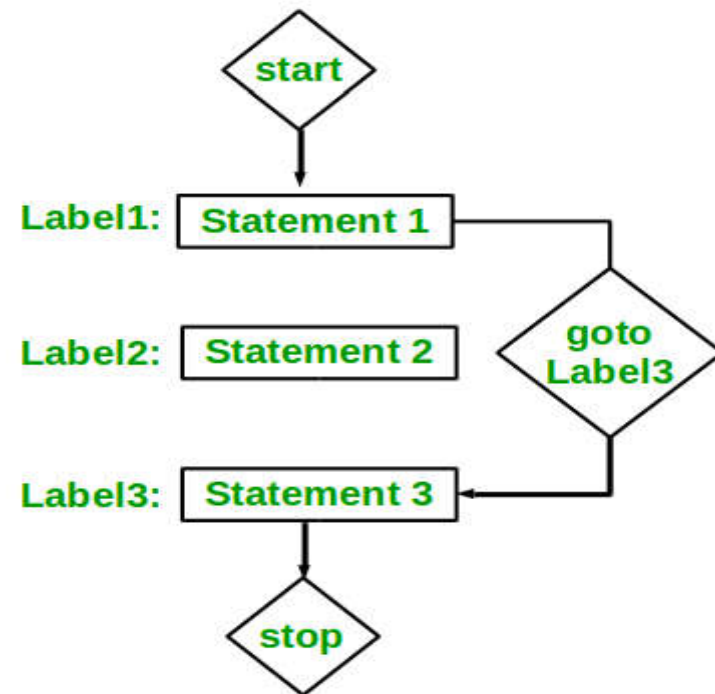
    while (n<5)
    {
        printf("%d ",n);
        n++;
        if(n==3) break;
        printf("%d ",n);
    }

    return 0;
}
```

► Exécution :

Ruptures de séquence : goto

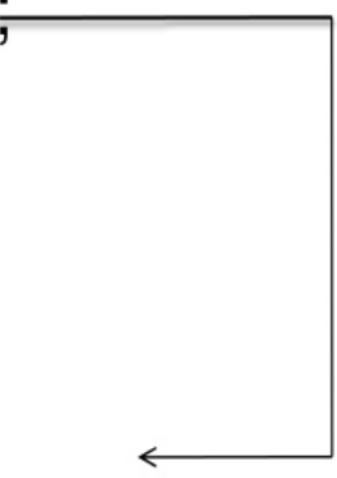
- ▶ Dans la programmation C, *goto* est utilisé pour modifier la séquence normale de l'exécution du programme en transférant le contrôle à une autre partie du programme
- ▶ **Flowchart: goto**



Ruptures de séquence : goto

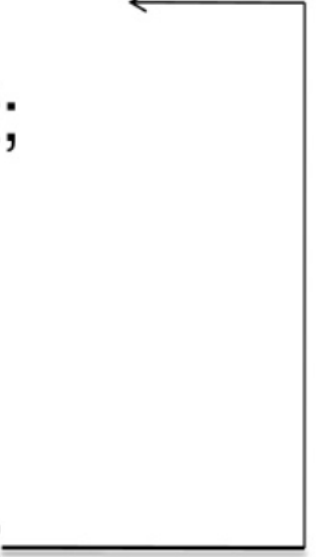
► Syntaxe: goto

```
goto label;  
-----  
-----  
-----  
label :  
Staement;
```



forword jump

```
label :  
Statement;  
-----  
-----  
-----  
goto label;
```



backward jump

Ruptures de séquence : goto

► Syntaxe: **goto** Forword jump

```
#include <stdio.h>
int main()
{
    int num;
    printf("Enter number \n");
    scanf("%d", &num);
    if (num > 0)
        printf("Number= %d \n", num);
    else
        // jump to war
        goto war;

    // some instructions

war:
    printf("End of program");

    return 0;
}
```

goto label;

label :
Staement;



forword jump

```
Enter number
10
Number= 10
End of program
```

```
Enter number
-10
End of program
```

Ruptures de séquence : goto

► Syntaxe: **goto Backward jump**

```
#include <stdio.h>
int main()
{   int n, i=0;

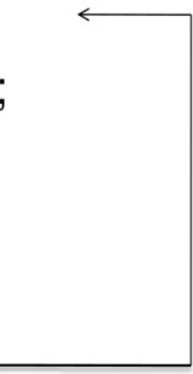
    printf("Enter a number: ");
    scanf("%d",&n);

    start:
    printf("%d\t",i);
    i++;
    if(i<n) goto start;

    return 0;
}
```

label :
Statement;

goto label;



backward jump

```
Enter a number: 6
0      1      2      3      4      5
```

Ruptures de séquence : goto

► Syntaxe: **goto Backward jump**

```
#include <stdio.h>
int main()
{   int n;

    start:
    printf("Enter a number: ");
    scanf("%d",&n);

    if(n<0) goto start;

    printf("Suite du programme ");

    return 0;
}
```

label :
Statement;

goto label;



backward jump

```
Enter a number: -2
Enter a number: -4
Enter a number: -5
Enter a number: 10
Suite du programme
```

Ruptures de séquence : goto

► goto

► Exemple :

```
1 #include <stdio.h>
2 int main()
3 {
4     int Totalmarks;
5
6     printf(" \n Please Enter your Subject Marks \n ");
7     scanf("%d", & Totalmarks);
8
9     if(Totalmarks >= 50)
10    {
11        goto Pass;
12    }
13    else
14        goto Fail;
15
16    Pass:
17        printf(" \n Congratulation! You made it \n");
18
19    Fail:
20        printf(" \n Better Luck Next Time \n");
21
22    return 0;
23 }
```

► Exécution :

```
Please Enter your Subject Marks
30

Better Luck Next Time
```

```
Please Enter your Subject Marks
88

Congratulation! You made it

Better Luck Next Time
```

Ruptures de séquence : goto

► goto

► Exécution :

► Exemple : return value;

```
1  #include <stdio.h>
2  int main()
3  {
4      int Totalmarks;
5
6      printf(" \n Please Enter your Subject Marks \n ");
7      scanf("%d", & Totalmarks);
8
9      if(Totalmarks >= 50)
10     {
11         goto Pass;
12     }
13     else
14         goto Fail;
15
16     Pass:
17     printf(" \n Congratulation! You made it \n"); return 1;
18
19     Fail:
20     printf(" \n Better Luck Next Time \n");
21
22     return 0;
23 }
```

```
Please Enter your Subject Marks
90

Congratulation! You made it
```

Ruptures de séquence : goto

► goto

► Exécution :

► Exemple : return value;

```
1  #include <stdio.h>
2  int main()
3  {
4      int Totalmarks;
5
6      printf(" \n Please Enter your Subject Marks \n ");
7      scanf("%d", & Totalmarks);
8
9      if(Totalmarks >= 50)
10     {
11         goto Pass;
12     }
13     else
14         goto Fail;
15
16     Pass:
17     printf(" \n Congratulation! You made it \n"); return 1;
18
19     Fail:
20     printf(" \n Better Luck Next Time \n");
21
22     return 0;
23 }
```

```
Please Enter your Subject Marks
90

Congratulation! You made it
```

Ruptures de séquence : goto

► goto

► Exemple : exit(value)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int Totalmarks;
7
8     printf(" \n Please Enter your Subject Marks \n ");
9     scanf("%d", & Totalmarks);
10
11     if(Totalmarks >= 50)
12     {
13         goto Pass;
14     }
15     else
16         goto Fail;
17
18     Pass:
19     printf(" \n Congratulation! You made it \n"); exit(1);
20
21     Fail:
22     printf(" \n Better Luck Next Time \n");
23
24     return 0;
25 }
```

► Exécution :

```
Please Enter your Subject Marks
88

Congratulation! You made it
```


Control statements in C

